

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 2

з навчальної дисципліни

«Технології розробки корпоративних Web-додатків»

Виконав:

Студент групи 403-ТН

Солонецький Роман Миколайович

Перевірив:

Мізік Андрій Олегович

Полтава 2026

Тема: Вебдодаток для відстеження звичок (HabitTracker)

Обґрунтування вибору технологічного стеку

Мова програмування Back-end: Node.js (у середовищі Next.js API Routes).

Фреймворк Front-end: React у складі фреймворку Next.js (Server-Side Rendering / Static Site Generation).

Система керування базами даних (СКБД): MongoDB (Нереляційна документоорієнтована БД).

Інструменти DevOps/Hosting: Платформа Vercel та GitHub для контролю версій.

Обрано екосистему Next.js (React + Node.js), оскільки вона дозволяє розробляти як клієнтську, так і серверну частини в єдиному репозиторії мовою JavaScript/TypeScript, що значно пришвидшує розробку.

Vercel забезпечує миттєве та безпроблемне розгортання таких додатків.

Обрано MongoDB, оскільки структура даних для відстеження звичок (де масиви виконаних днів постійно зростають) є гнучкою, і NoSQL підхід дозволяє ефективно працювати з такими JSON документами без складних міграцій таблиць.

Архітектурне моделювання за методологією C4

Діаграма System Context відображає систему HabitTracker на найвищому рівні як єдиний механізм, з яким взаємодіє користувач для управління своїми звичками. Також показано інтеграцію із зовнішнім сервісом (Email Service), який необхідний для реалізації функції відновлення пароля.

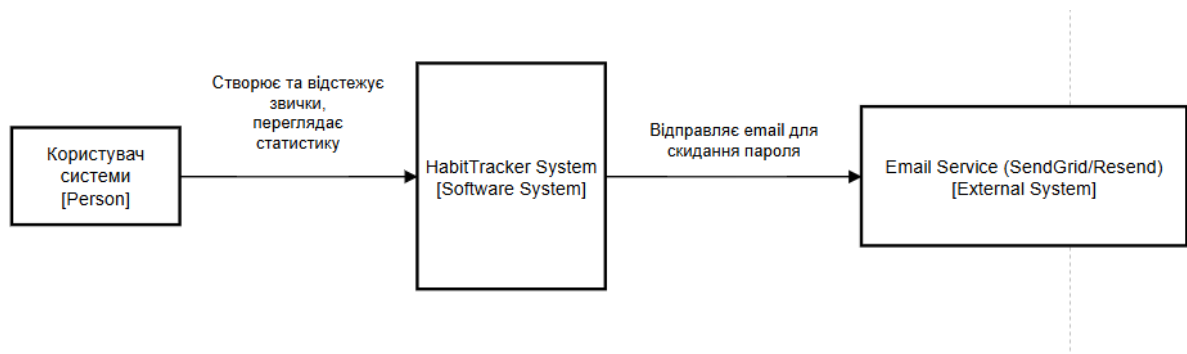


Рис. 1 – System Context Diagram

На цій діаграмі розкрито внутрішню архітектуру системи, яка складається з трьох основних контейнерів: клієнтського вебдодатка (React), серверного API (Next.js/Node.js) та хмарної бази даних (MongoDB). Вона демонструє, як клієнтська частина надсилає HTTPS-запити до бекенду, який, у свою чергу, виконує операції читання та запису в базі даних.

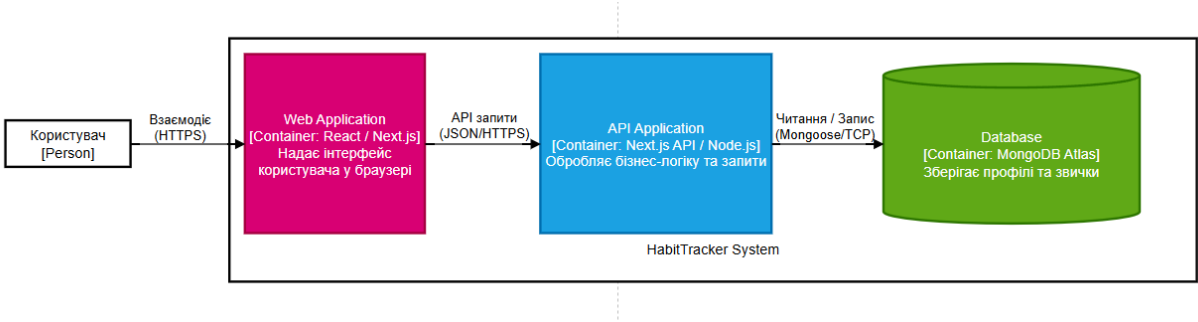


Рис. 2 – Container Diagram

Діаграма деталізує архітектуру серверного контейнера (API Application), розділяючи його на логічні компоненти. Показано взаємодію контролерів (AuthController, HabitController), які обробляють вхідні запити, із відповідними репозиторіями моделей (UserRepository, HabitRepository) для безпосередньої роботи з базою даних.

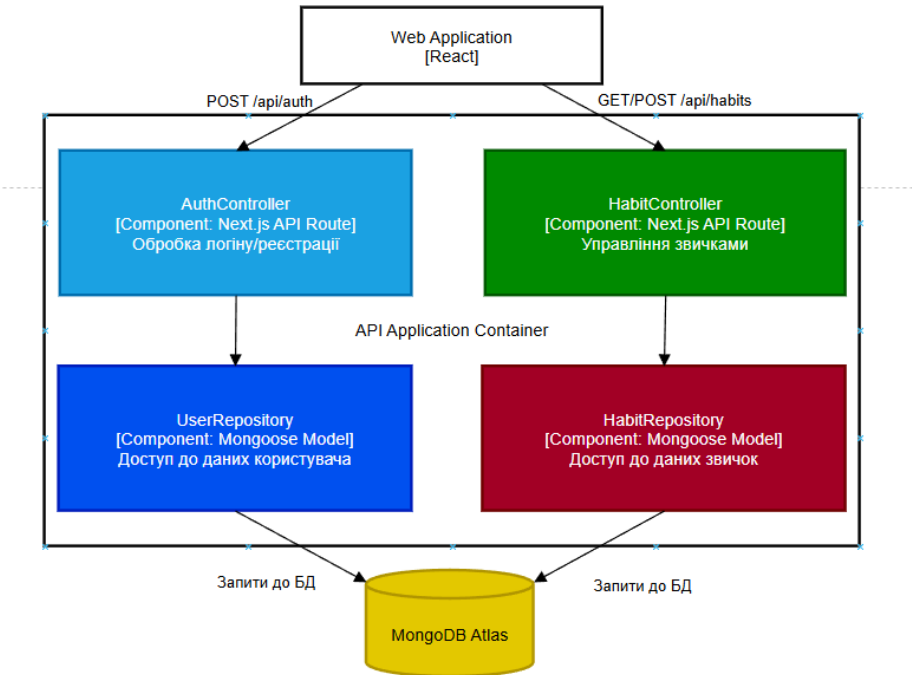


Рис. 3 – Component Diagram

Проектування бази даних (NoSQL)

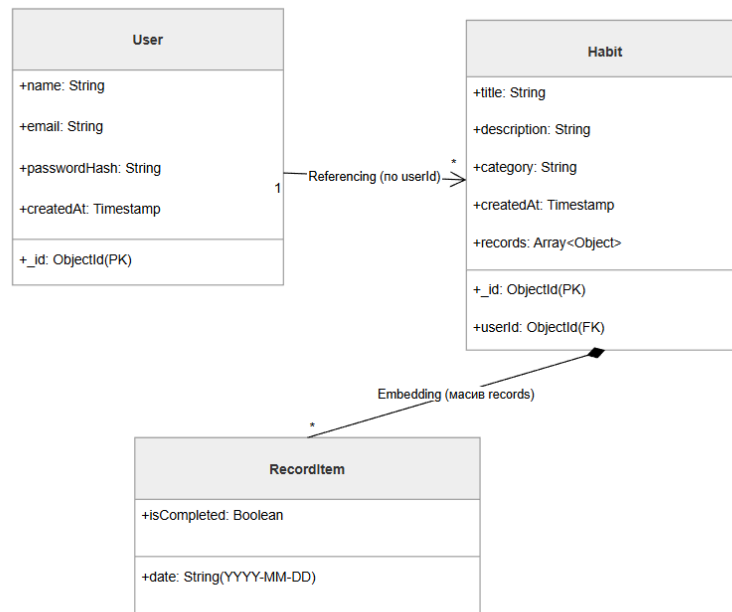


Рис. 4 – Схема бази даних



Рис. 5-6 – Документи в колекціях habits та users

Referencing (Посилання): Використано стратегію Referencing для зв'язку між Користувачем та Звичками (збереження `userId` у документі звички). Обґрунтування: Це дозволяє уникнути переповнення документа користувача, оскільки кількість звичок може бути великою.

Embedding (Вкладення): Використано стратегію Embedding для зберігання історії виконання (`completedDates` як масив дат всередині документа звички). Обґрунтування: Оскільки історія відміток завжди потрібна разом із самою звичкою для відображення графіка прогресу, вкладення дозволяє отримати всі дані за один запит.

Контрольні питання

1. Що таке модель C4 і для чого потрібен кожен з її рівнів?

Це підхід до візуалізації архітектури.

Context - загальний погляд (система і користувачі).

Container - архітектура додатків (фронтенд, бекенд, БД).

Component - структура коду всередині одного контейнера (контролери, сервіси).

2. Різниця між SQL та NoSQL? Коли що обирати?

SQL зберігає дані у строгих таблицях зі зв'язками (добре для фінансів, де важлива структура). NoSQL (як MongoDB) зберігає дані у гнучких документах (JSON).

3. Що таке теорема CAP?

Теорема стверджує, що розподілена система може одночасно забезпечувати лише дві з трьох властивостей: узгодженість даних (Consistency), доступність (Availability) та стійкість до розділення мережі (Partition tolerance). Це допомагає зрозуміти компроміси при виборі бази даних.

4. Переваги та недоліки мікросервісів порівняно з монолітом?

Моноліт: легко розробляти на старті, але важко масштабувати і оновлювати великий проєкт.

Мікросервіси: кожен модуль працює окремо (легко масштабувати окремі частини, можна використовувати різні мови), але система стає складною у розгортанні та тестуванні.